Rockchip RKStudio Third-Party Algorithm Development Guide

ID: RK-KF-SF-044

Release Version: 2.0.0

Release Date: 2025-03-11

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.commailto:fae@rock-chips.com)

Preface

Overview

This document aims to elaborate on the specific methods for importing third-party algorithms into RKStudio_Tool, assisting developers in implementing the full-process operations such as algorithm access, debugging, and deployment, and providing clear guidance and specifications for algorithm integration.

Product Version

Chipset
RK2118

Intended Audience

This document (this guide) is mainly intended for:

Audio engineers

Technical support engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Huaping Liao	2024-05-23	Initial version
V2.0.0	Huaping Liao	2025-03-11	Modified interfaces and calling methods

Contents

- Rockchip RKStudio Third-Party Algorithm Development Guide
  - RKStudio Tool Interface Configuration
    - File Description
    - Framework Differences
    - JSON File Common Configuration
    - V1 Param Introduction
    - V2 Param Introduction
    - Multi-Algorithm Configuration and Validation Rules
    - Configuration Notes
  - Algorithm Interfaces
    - BlockPro
    - BlockAlgo
    - Third-Party Algorithm Interface Registration
  - General Interfaces
    - WAV Partition File Management
  - Appendix: Registration Code Example

RKStudio Tool Interface Configuration

File Description

In the CustomAlgorithms directory of the tool, there are two versions of algorithm parameter configuration files:

- custom_algo_v1.json: Written based on the v1 framework (applicable to frameworks prior to document version V2.0.0).
- custom_algo_v2.json: Written based on the v2 framework.

Framework Differences

- **v1 Framework**
  - Some attribute names have changed but are backward compatible with older versions.
  - The JSON configuration is relatively simple, and the configuration interface is presented as a list of controls, **suitable for simple parameter configuration scenarios**.
- **v2 Framework**
  - The JSON configuration is relatively complex, and the configuration interface adopts a layout with multiple tabs and controls, **suitable for complex parameter configuration scenarios**.

Developers can select the matching framework version for configuration according to actual needs.

JSON File Common Configuration

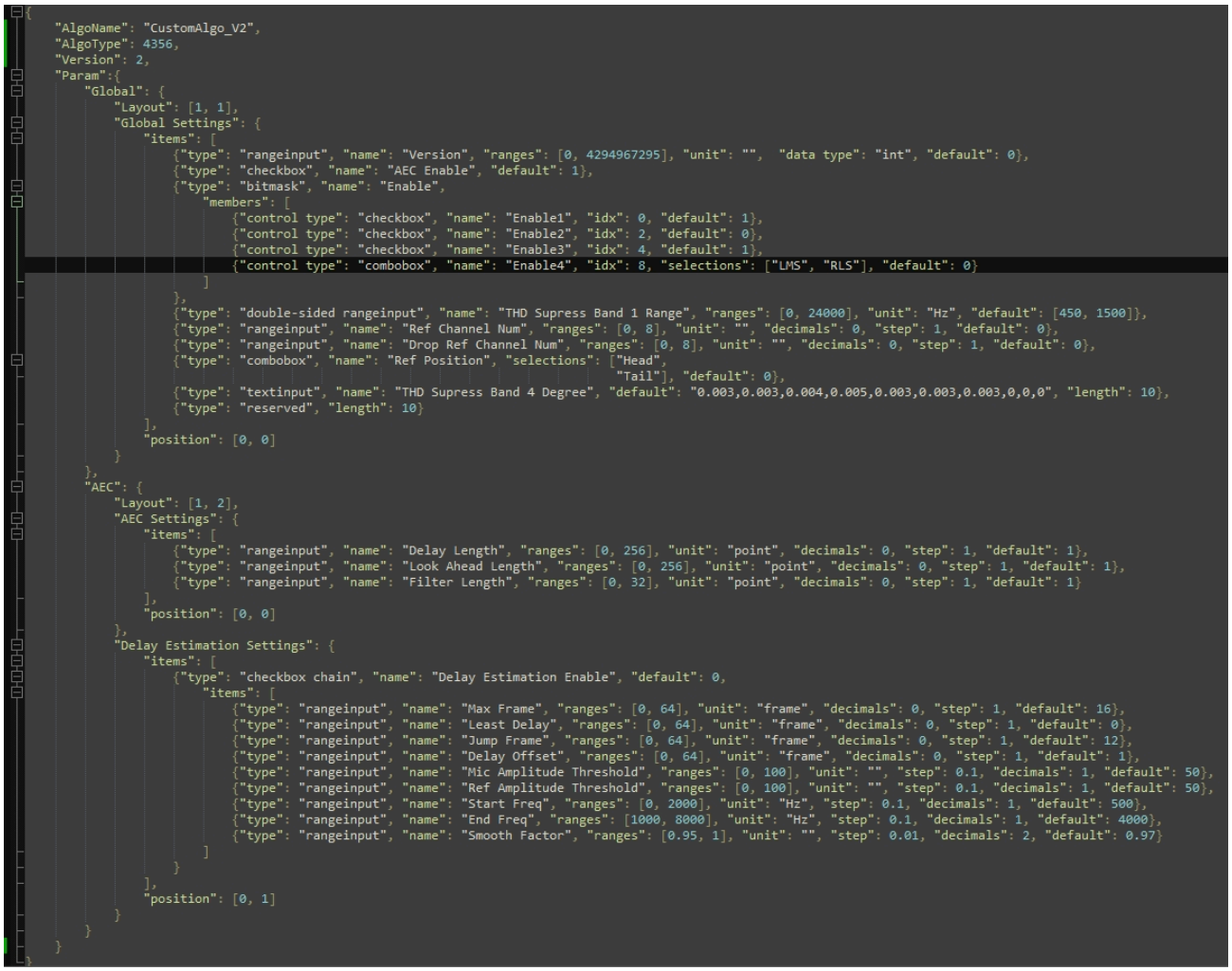
- `AlgoName` : The algorithm name, used as the identifier displayed in the control bar. It must be globally unique and must not contain spaces or other special characters.
- `AlgoType` : The algorithm type identifier, used to match the `algoType` when registering the board-side algorithm function. It must be globally unique. The value range of custom algorithm types is strictly limited to `0x1100~0x11FF` . For details on algorithm registration, see Chapter 2.3.
- `Version` : The JSON framework version, configured as 1 or 2. If not configured, the default version is 1.
- `DataType` : The parameter data type, configured as int/int32/int32_t/uint/uint32/uint32_t/float. If not configured, the default data type is float.
- `Param` : Both v1 and v2 JSON frameworks have the `Param` attribute, which contains parameter configurations related to the algorithm. Each attribute corresponds to a parameter position, supporting data types such as int/int32/int32_t/uint/uint32/uint32_t/float, with each parameter having a data width of 4 bytes. The finally generated parameters are sequentially placed in the

algorithm's parameters in the order defined in the JSON. That is, when the algorithm calls the registered function, the algorithm's parameters can be obtained from the `algoSt->pParam` pointer, and `algoSt->nParamSize` corresponds to the size of the parameter memory in bytes.



```
{
    "AlgoName": "CustomAlgo_V1",
    "AlgoType": 4353,
    "Version": 1,
    "DataType": "float",
    "Param": {
        "Type": {
            "type": "combo",
            "selections": ["GeneralLP", "GeneralHP"],
            "default": 0
        },
        "Gain (dB)": {
            "type": "input",
            "min": -20,
            "max": 20,
            "decimals": 2,
            "step": 0.01,
            "default": 0
        },
        "Fc (Hz)": {
            "type": "reserved"
        },
        "Q": {
            "type": "input",
            "min": 0.01,
            "max": 16,
            "decimals": 3,
            "step": 0.001,
            "default": 1.41
        },
        "Value": {
            "type": "input",
            "data_type": "int",
            "min": 1,
            "max": 16,
            "default": 3
        }
    }
}
```

Figure 1-1 V1 JSON File



```
{
    "AlgoName": "CustomAlgo_V2",
    "AlgoType": 4356,
    "Version": 2,
    "Param":{
        "Global": {
            "Layout": [1, 1],
            "Global Settings": {
                "items": [
                    {"type": "rangeinput", "name": "Version", "ranges": [0, 4294967295], "unit": "",  "data type": "int", "default": 0},
                    {"type": "checkbox", "name": "AEC Enable", "default": 1},
                    {"type": "bitmask", "name": "Enable",
                        "members": [
                            {"control type": "checkbox", "name": "Enable1", "idx": 0, "default": 1},
                            {"control type": "checkbox", "name": "Enable2", "idx": 2, "default": 0},
                            {"control type": "checkbox", "name": "Enable3", "idx": 4, "default": 1},
                            {"control type": "combobox", "name": "Enable4", "idx": 8, "selections": ["LMS", "RLS"], "default": 0}
                        ]
                    },
                    {"type": "double-sided rangeinput", "name": "THD Supress Band 1 Range", "ranges": [0, 24000], "unit": "Hz", "default": [450, 1500]},
                    {"type": "rangeinput", "name": "Ref Channel Num", "ranges": [0, 8], "unit": "", "decimals": 0, "step": 1, "default": 0},
                    {"type": "rangeinput", "name": "Drop Ref Channel Num", "ranges": [0, 8], "unit": "", "decimals": 0, "step": 1, "default": 0},
                    {"type": "combobox", "name": "Ref Position", "selections": ["Head",
                                                                                "Tail"], "default": 0},
                    {"type": "textinput", "name": "THD Supress Band 4 Degree", "default": "0.003,0.003,0.004,0.005,0.003,0.003,0.003,0,0,0", "length": 10},
                    {"type": "reserved", "length": 10}
                ],
                "position": [0, 0]
            }
        },
        "AEC": {
            "Layout": [1, 2],
            "AEC Settings": {
                "items": [
                    {"type": "rangeinput", "name": "Delay Length", "ranges": [0, 256], "unit": "point", "decimals": 0, "step": 1, "default": 1},
                    {"type": "rangeinput", "name": "Look Ahead Length", "ranges": [0, 256], "unit": "point", "decimals": 0, "step": 1, "default": 1},
                    {"type": "rangeinput", "name": "Filter Length", "ranges": [0, 32], "unit": "point", "decimals": 0, "step": 1, "default": 1}
                ],
                "position": [0, 0]
            },
            "Delay Estimation Settings": {
                "items": [
                    {"type": "checkbox chain", "name": "Delay Estimation Enable", "default": 0,
                        "items": [
                            {"type": "rangeinput", "name": "Max Frame", "ranges": [0, 64], "unit": "frame", "decimals": 0, "step": 1, "default": 16},
                            {"type": "rangeinput", "name": "Least Delay", "ranges": [0, 64], "unit": "frame", "decimals": 0, "step": 1, "default": 0},
                            {"type": "rangeinput", "name": "Jump Frame", "ranges": [0, 64], "unit": "frame", "decimals": 0, "step": 1, "default": 12},
                            {"type": "rangeinput", "name": "Delay Offset", "ranges": [0, 64], "unit": "frame", "decimals": 0, "step": 1, "default": 1},
                            {"type": "rangeinput", "name": "Mic Amplitude Threshold", "ranges": [0, 100], "unit": "", "step": 0.1, "decimals": 1, "default": 50},
                            {"type": "rangeinput", "name": "Ref Amplitude Threshold", "ranges": [0, 100], "unit": "", "step": 0.1, "decimals": 1, "default": 50},
                            {"type": "rangeinput", "name": "Start Freq", "ranges": [0, 2000], "unit": "Hz", "step": 0.1, "decimals": 1, "default": 500},
                            {"type": "rangeinput", "name": "End Freq", "ranges": [1000, 8000], "unit": "Hz", "step": 0.1, "decimals": 1, "default": 4000},
                            {"type": "rangeinput", "name": "Smooth Factor", "ranges": [0.95, 1], "unit": "", "step": 0.01, "decimals": 2, "default": 0.97}
                        ]
                    }
                ],
                "position": [0, 1]
            }
        }
    }
}
```


Figure 1-2 V2 JSON File

V1 Param Introduction

Taking custom_algo_v1.json as an example, the configured attribute names correspond to the parameter names in the configuration menu. For example, the first name `Type` corresponds to the "Type" in the UI configuration table. Each parameter has the following attributes:

- `type` : The UI interaction type for input:
  - `combobox` : Combobox form, supporting multiple preset values. The list is stored in `"selections": ["GeneralLP", "GeneralHP"]`. `default` is the index corresponding to the default value, and the passed parameter value is the corresponding index. For example, if "GeneralLP" is selected, the passed parameter value is 0.
  - `rangecombo` : Automatically generates a `selections` list based on the set "min" and "max" values. The values in the list are evenly distributed between min and max. `default` is the index corresponding to the default value, and the passed parameter value is the corresponding index.

    - `rangeinput` : Input box form, allowing input of values within the "min" and "max" range. `default` is the default value.
    - `reserved` : Only used as a parameter placeholder, with no functionality and not displayed in the configuration box. It is convenient for discontinuous parameter spaces or scenarios where user configuration is not required. The default value is 0.
- `data_type` : The data type. If not configured, the `DataType` configured in 1.3 is used. If `DataType` is not configured, float is used by default. Supported data types are int/int32/int32_t/uint/uint32/uint32_t/float.
- `min` : The minimum value, required for `rangeinput` and `rangecombo` types.
- `max` : The maximum value, required for `rangeinput` and `rangecombo` types.
- `decimals` : The number of decimal places, valid for `rangeinput` type. If not configured, the default is 2.
- `step` : The step size for each change when switching values with the up/down keys, valid for `rangeinput` type. If not set, the default is 0.01.
- `selections` : The option list for the combobox, required for `combobox` type.

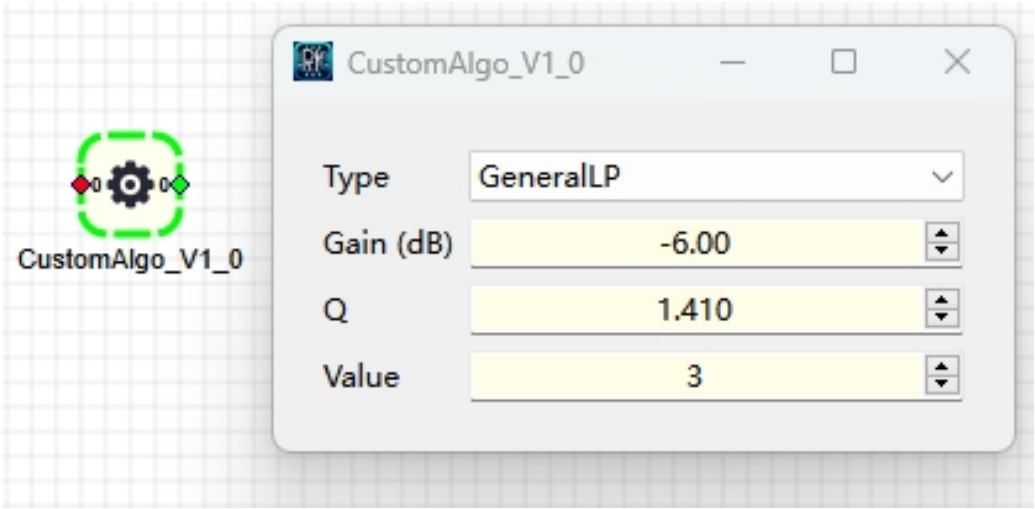


Figure 1-3 Configuration Interface Corresponding to custom_algo_v1.json

V2 Param Introduction

Taking custom_algo_v2.json as an example, **`Global`** and **`AEC`** correspond to two tabs, each with multiple controls, and each control corresponds to a parameter. The attributes of each parameter in the AEC tab are as follows:

- `Layout` : [1, 2] indicates that the layout of the current AEC page is 1 row and 2 columns.
- `AEC Settings` and `Delay Estimation Settings` represent two different layout names.
    - `position` : [0, 0] indicates that the layout is located at row 0, column 0. If not configured, the default position is [0, 0].

- `items` defines control parameters.

Taking `Delay Estimation Settings` as an example, each parameter has the following attributes:

- `type` : The UI interaction type for input:
  - `checkbox chain` : Checkbox chain. If checked, all subsequent "items" are displayed; otherwise, all are hidden. 1 if checked, 0 otherwise.
  - `combobo` : Combobox form, allowing selection of multiple preset values. The list is stored in `"selections": ["LMS", "RLS"]` . `default` is the index corresponding to the default value, and the passed parameter value is the corresponding index. For example, if "LMS" is selected, the passed parameter value is 0.
  - `rangecombo` : Automatically generates a `selections` list based on the set "min" and "max" values. The values in the list are evenly distributed between min and max. `default` is the index corresponding to the default value, and the passed parameter value is the corresponding index.
  - `rangeinput` : Input box form, allowing input of values within the "min" and "max" range. `default` is the default value.
  - `double-sided rangeinput` : Displays two input boxes. Except for `default` , other configurations are shared. Generally used for setting paired range values, outputting two parameters. Other usages are the same as `rangeinput` .
  - `reserved` : Only used as a parameter placeholder, with no functionality and not displayed in the configuration box. It is convenient for discontinuous parameter spaces or scenarios where user configuration is not required. The default value is 0, and the data length is `length` .
  - `checkbox` : Checkbox form, 1 if checked, 0 otherwise.
  - `textinput` : Allows importing corresponding data from a text file, with the data length being `length` .
  - `bitmask` : Defines a parameter type that can configure 32 bits. The data type is fixed as uint32 and does not support custom configuration.、
  - `members` : Defines the option list, with the following related attributes:
  - `control type` : Can be configured as `checkbox` or `combobox` , corresponding to checkbox and combobox respectively. The combobox can only have two configurations, corresponding to 0 and 1.
  - `name` : The option name.
  - `default` : The default value, 0 or 1.
  - `idx` : The corresponding bit index, ranging from 0 to 31. If selected, the bit value is 1; otherwise, 0.
  - `selections` : The option list for the combobox, required when `control type` is `combobox` , supporting only two configuration items.

- `data_type` : The transmitted data type. If not configured, the `DataType` configured in 1.3 is used. If `DataType` is not configured, float is used by default. Supported data types are int/int32/int32_t/uint/uint32/uint32_t/float.
- `min` : The minimum value, required for `rangeinput` and `rangecombo` types.
- `max` : The maximum value, required for `rangeinput` and `rangecombo` types.
- `decimals` : The number of decimal places, valid for `rangeinput` type. If not configured, the default is 2.
- `step` : The step size for each change when switching values with the up/down keys, valid for `rangeinput` type. If not set, the default is 0.01.
- `selections` : The option list for the combobox, required for `combobox` type.

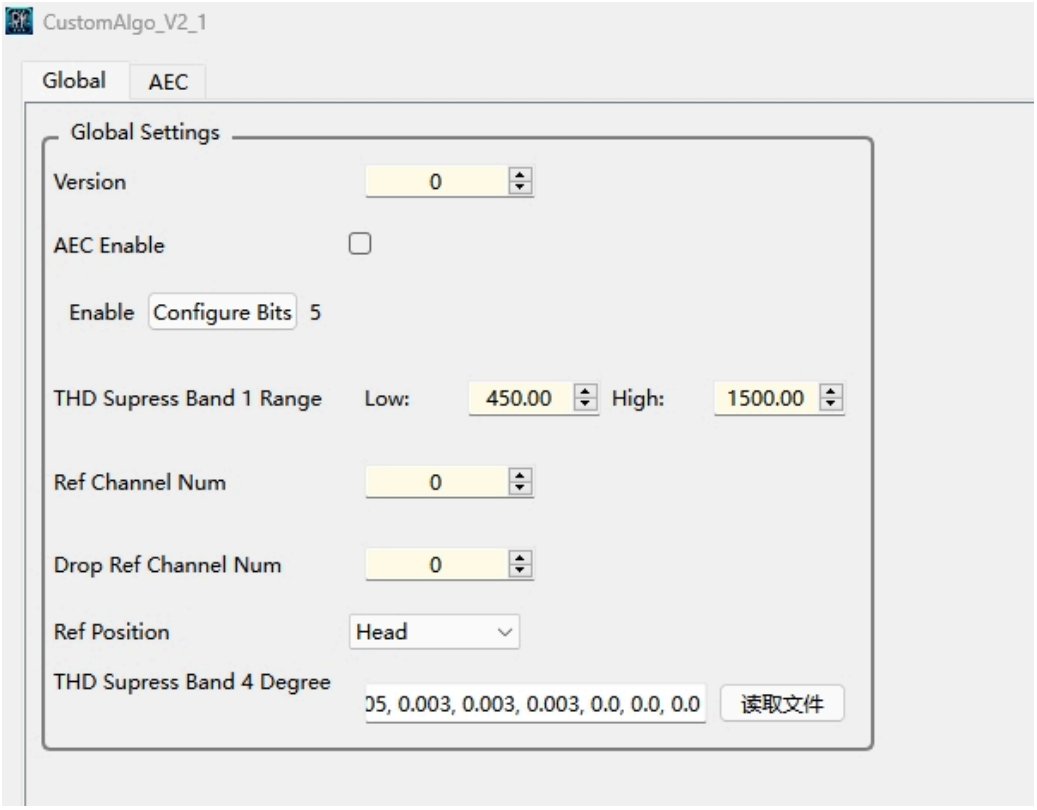


Figure 1-4 Configuration Interface Corresponding to custom_algo_v2.json

Multi-Algorithm Configuration and Validation Rules

- If multiple algorithms exist, multiple JSON files can be added, and they will take effect after reopening the project diagram.
- Newly added algorithm controls/modified algorithm names and types take effect only after reopening the tool.
- Parameter content supports dynamic modification, taking effect immediately after re-dragging the control, without needing to reopen the tool.

Configuration Notes

- The same algorithm control can be dragged multiple times, but the control name cannot be customized, and the main body of the control name must be consistent with the `AlgoName` attribute.
- The defined JSON file must strictly follow JSON format requirements; otherwise, the tool will report an error. You can verify the correctness of the file format through https://www.json.cn/.
- The names of each attribute must be filled in strictly as described in the document; otherwise, the algorithm control will fail to be added.

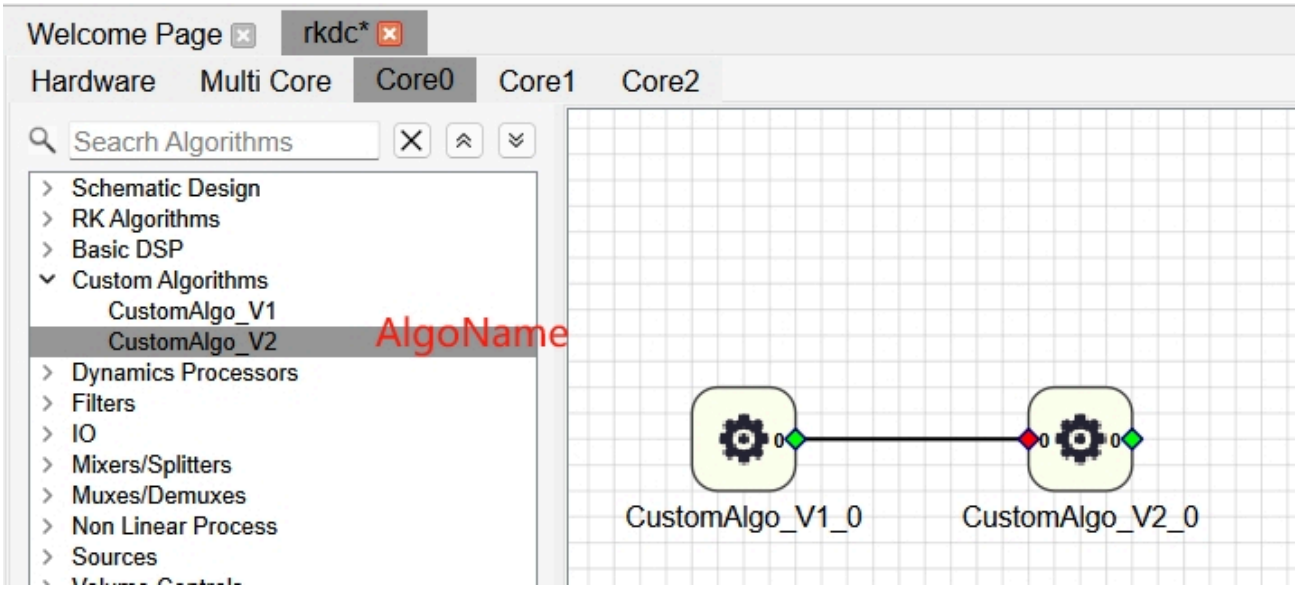


Figure 1-5 Position of the Control in the Toolbar

Algorithm Interfaces

Currently, custom algorithm interfaces are supported. Developers need to encapsulate the interfaces, compile them by themselves, and register them into RKStudio algorithms. **The structures, function names, parameter types, and related usages introduced in V1.0.0 of this document have changed. Please refer to the description of V2.0.0 for modifications to avoid compilation and usage issues**.

BlockPro

The `BlockPro` structure is shared by all algorithms and exists in `BlockAlgo->pBlockPro`. Common members include sampling rate and frame length. Related functions are for initializing and destroying `BlockPro`. Generally, initializing the diagram will call the initialization function once, passing `BlockPro`

to each algorithm module. The interface calls are integrated into the RKStudio project, so there is no need to implement or call them by yourself. Only the usage is shown here.

```
BlockPro structure
typedef struct _BlockPro
{
    int             nSamplingRate;  /* sample rate of the audio block */
    int             bitRate;        /* bit rate of the audio block, no used now */
    int             nBlockSize;     /* block size of the audio block */
    float          *tmpBuffer;     /* 2 * nBlockSize * sizeof(float) bytes temporary buffer */
    FadeWinSt      *fadeWin;       /* fade in and out window */
    unsigned int    toolVersion;

    int             nReserve0;      /* reserved */
    int             nReserve1;      /* reserved */
} BlockPro;
```

- `nSamplingRate` : The sampling rate of the current algorithm in Hz, which uses the sampling rate configured on the RkStudioTool configuration bar.
- `bitRate` : The bit width of the current algorithm, which uses the bit width configured on the RkStudioTool configuration bar.
- `nBlockSize` : The frame length of the current algorithm, which uses the frame length configured on the RkStudioTool configuration bar.
- `*tmpBuffer` : Points to a temporary buffer that can be temporarily called by all algorithms. The size is 2 * nBlockSize * sizeof(float).
- `*fadeWin` : The fade-in and fade-out window, generally used for fade-in and fade-out processing of the algorithm.
- `toolVersion` : The version number of RkStudioTool, used for version compatibility.
- `nReserve0` : Reserved member, not used temporarily.
- `nReserve1` : Reserved member, not used temporarily.

```
BlockPro related initialization and destruction functions
BlockPro *RKST_ALGO_InitBlock(int nSamplingRate,
                              int bitRate,
                              int nBlockSize,
                              FadeWinSt *fadeWin,
                              uint32_t toolVersion);
void RKST_ALGO_DestroyBlock(BlockPro *st);
```

BlockAlgo

`BlockAlgo` is a structure shared by algorithm calling interfaces.

```
BlockAlgo structure
typedef struct _BlockAlgo
{
    int         nInputs;          /* number of input pins */
    int         nOutputs;         /* number of output pins */
    BlockPro   *pBlockPro;        /* pointer to the array of pBlockPro common structure */
    int         methodType;

    float      *pParam;           /* pointer to the parameter memory */
    int         nParamSize;       /* size of the parameter memory, unit is Byte */
    void       *pAlgoCore;        /* pointer to the algorithm memory */
} BlockAlgo;
```

- `nInputs` : Number of input channels of the algorithm.
- `nOutputs` : Number of output channels of the algorithm.
- `*pBlockPro` : Pointer to the `BlockPro` structure, used for global parameter transfer. For structure members, see Chapter 2.1.
- `*methodType` : The current algorithm type, defined in `RK_TYPE` in the `rkstudio_process.h` file.
- `*pParam` : Parameters used by the current algorithm, automatically allocated and managed by the framework. When calling the algorithm interface, the relevant pointers and contents have been allocated.
- `nParamSize` : The size of the parameter memory in bytes.
- `*pAlgoCore` : Pointer to the algorithm core memory. If the developer defines the `GetAlgoSize` function, the framework automatically allocates the corresponding space. If not defined, the space needs to be allocated and freed by yourself.

```
int RKST_ALGO_GetAlgoSize(BlockAlgo *algoSt);
int RKST_ALGO_Init(BlockAlgo *algoSt);
int RKST_ALGO_Destroy(BlockAlgo *algoSt);
int RKST_ALGO_SetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t nParamSize);
int RKST_ALGO_GetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t *nParamSize);
int RKST_ALGO_Process(BlockAlgo *algoSt,
                      float **dataIn,
                      float **dataOut);
```

- `RKST_ALGO_GetAlgoSize` : Obtains the space size required for `pAlgoCore` in bytes. If no external space allocation is needed, the return value is 0. If this function is not registered, the framework does not allocate space by default. `*pAlgoCore` can be operated by the developer.
- `RKST_ALGO_Init` : Used to perform algorithm initialization. When called, all members of the `BlockAlgo` structure have been defined, and `*pParam` and `*pAlgoCore` have been allocated (if `GetAlgoSize` returns 0, `pAlgoCore` needs to be allocated by the user and manually freed during destruction).
- `BlockAlgo_Destroy` : Frees the space allocated during initialization.
- `RKST_ALGO_SetParam` : Used to dynamically set parameters. `nParamOff` is the offset of the parameter in `pParam` (each offset is 4 bytes), and `nParamSize` is the parameter size in bytes. The framework automatically judges the parameter address range and corrects `nParamSize` to ensure it does not exceed the size of `pParam` .
- `RKST_ALGO_GetParam` : Used to dynamically obtain parameters. `nParamOff` is the offset of the parameter in `pParam` (each offset is 4 bytes), and `nParamSize` is the parameter size in bytes. The framework automatically judges the parameter address range, corrects `nParamSize` to ensure it does not exceed the size of `pParam` , and sets the actual `nParamSize` value.
- `RKST_ALGO_Process` : The algorithm execution function. `dataIn` and `dataOut` are input and output data. The data structure is a two-dimensional pointer. For example, with 2 channels, `dataIn[0]` points to the first channel data, `dataIn[1]` points to the second channel data, and all channel data are stored independently (deinterleaved form). To save memory and avoid data copying, `dataOut` may reuse the address space of `dataIn` . If the algorithm needs to call `dataIn` multiple times internally, ensure that the original data of `dataIn` is not used after assigning values to `dataOut` -related data.

Third-Party Algorithm Interface Registration

Call the `RKST_ALGO_Register` function to complete algorithm registration. **Registration is allowed only once during the entire program lifecycle**. `algoType` is the algorithm type, which must be consistent with `algoType` in the custom Json file of the tool. Its value range is strictly limited to `0x1100~0x11FF` and must be unique to distinguish different algorithms. For the definition and function of the function, see Chapter 2.3. `algoGetAlgoSize` is an optional function. If this function is not registered, the framework does not allocate `*pAlgoCore` space by default. When not registered, the corresponding member of the structure must be set to `NULL` ; otherwise, pointer exceptions may occur.

```
typedef struct
{
    int algoType;
    int (*algoInit)(BlockAlgo *algoSt);
    int (*algoSetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t nParamSize);
    int (*algoGetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t* nParamSize);
    int (*algoGetAlgoSize)(BlockAlgo *algoSt);
    int (*algoProcess)(BlockAlgo *algoSt,
                       float **dataIn,
                       float **dataOut);
    int (*algoDestroy)(BlockAlgo *algoSt);
} RkStudioFunc;

int RKST_ALGO_Register(RkStudioFunc* func);
```

General Interfaces

WAV Partition File Management

In Tool --> File Partition Configuration, multiple files are supported for configuration, and each file corresponds to an Idx. After file partition configuration, an `rkwav.img` partition file is generated, and the partition file is re-flashed into the flash. In the algorithm interface, the file address and size can be obtained by calling the public interface.

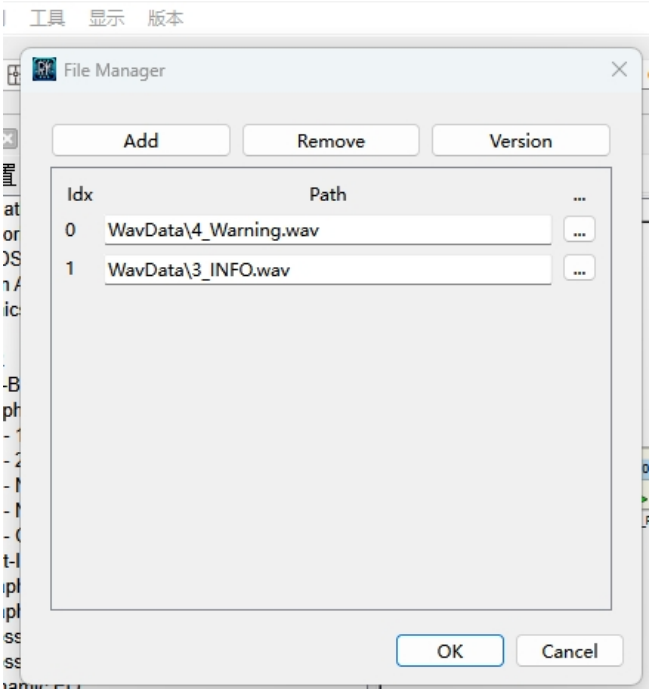


Figure 3-1 WAV Partition File Management Interface

```
int RKST_WAV_GetAddr(uint32_t fileIdx, void **addr, uint32_t *size);
```

`fileIdx` indicates the idx of the file, `addr` indicates the file address, and `size` indicates the file size. A return value of 0 indicates success, and other values indicate that the file does not exist.

Example code: Obtain the address and size of the file with idx 0 through the interface. After obtaining the pointer, access the file content as memory.

```
void *addr;
uint32_t size;
int ret = RKST_WAV_GetAddr(0, &addr, &size);
```

Appendix: Registration Code Example

The relevant structure and interface definitions are in `rkstudio/basic/librkstudio_common.h` and `rkstudio/interface/rkstudio_process.h` of the project.

```
typedef struct _BlockPro
{
    int            nSamplingRate;  /* sample rate of the audio block */
    int            bitRate;        /* bit rate of the audio block, no used now */
    int            nBlockSize;     /* block size of the audio block */
    float         *tmpBuffer;      /* 2 * nBlockSize * sizeof(float) bytes temporary buffer */
    FadeWinSt     *fadeWin;        /* fade in and out window */
    unsigned int   toolVersion;    /* RkStudioTool version */

    int            nReserve0;      /* reserved */
    int            nReserve1;      /* reserved */
} BlockPro;

#define RKST_ALIGN    8
#define RKST_ALIGN_UP(x) (((x) + (RKST_ALIGN -1)) & ~(RKST_ALIGN -1))
#if defined(_MSC_VER)
#define __RKST_ADDR_ALIGN__  __declspec(align(RKST_ALIGN))
#elif defined(__XTENSA__)
#define __RKST_ADDR_ALIGN__ __attribute__((aligned(RKST_ALIGN)))
#else
#define __RKST_ADDR_ALIGN__ ;
#endif

#pragma pack(push, RKST_ALIGN)
typedef struct _BlockAlgo
{
    int         nInputs;          /* number of input pins */
    int         nOutputs;         /* number of output pins */
    BlockPro   *pBlockPro;        /* pointer to the array of pBlockPro common structure */
    int         methodType;

    float      *pParam;           /* pointer to the parameter memory */
    int         nParamSize;       /* size of the parameter memory, unit is Byte */
    void       *pAlgoCore;        /* pointer to the algorithm memory */
} BlockAlgo;
#pragma pack(pop)


typedef struct
{
    int algoType;
    int (*algoInit)(BlockAlgo *algoSt);
    int (*algoSetParam)(BlockAlgo* algoSt,
```

```
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t nParamSize);
    int (*algoGetParam)(BlockAlgo* algoSt,
                        void* pParam,
                        uint32_t nParamOff,
                        uint32_t* nParamSize);
    int (*algoGetAlgoSize)(BlockAlgo *algoSt);
    int (*algoProcess)(BlockAlgo *algoSt,
                       float **dataIn,
                       float **dataOut);
    int (*algoDestroy)(BlockAlgo *algoSt);
} RkStudioFunc;

int RKST_ALGO_GetVersion(uint16_t *major,
                         uint16_t *minor,
                         const char **date,
                         const char **time);
BlockPro *RKST_ALGO_InitBlock(int nSamplingRate,
                              int bitRate,
                              int nBlockSize,
                              FadeWinSt *fadeWin,
                              uint32_t toolVersion);
void RKST_ALGO_DestroyBlock(BlockPro *blockProSt);

int RKST_ALGO_Register(RkStudioFunc* func);
int RKST_ALGO_GetAlgoSize(BlockAlgo *algoSt);
int RKST_ALGO_Init(BlockAlgo *algoSt);
int RKST_ALGO_Destroy(BlockAlgo *algoSt);
int RKST_ALGO_SetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t nParamSize);
int RKST_ALGO_GetParam(BlockAlgo* algoSt,
                       void *pParam,
                       uint32_t nParamOff,
                       uint32_t *nParamSize);
int RKST_ALGO_Process(BlockAlgo *algoSt,
                      float **dataIn,
                      float **dataOut);
```

The registration code can be found in `rkstudio/librkstudio_custom.c` of the project.

```c
typedef struct
{
    float gain;
    // todo
} CustomAlgoSt;

int Custom_Algo_SetParam(BlockAlgo* algoSt, void* param, uint32_t nParamOff, uint32_t nParamSize
{
    int chanNum = algoSt->nOutputs;
    float *pParam = (float *)algoSt->pParam;
    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;

    if (param)
    {
        memcpy(pParam + nParamOff, param, nParamSize);
    }
    st->gain = pParam[1];
    // todo use the pParam
    return 0;
}

int Custom_Algo_GetAlgoSize(BlockAlgo *algoSt)
{
    return sizeof(CustomAlgoSt);
}

int Custom_Algo_Init(BlockAlgo *algoSt)
{

    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;

    // algoSt->pParam ready now, don't need care
    return Custom_Algo_SetParam(algoSt, NULL, 0, 0);
}

int Custom_Algo_Process(BlockAlgo *algoSt, float **dataIn, float **dataOut)
{
    CustomAlgoSt *st = (CustomAlgoSt *)algoSt->pAlgoCore;
    int chanNum = algoSt->nInputs;
    int blockSize = algoSt->pBlockPro->nBlockSize;
    for (int chan = 0; chan < chanNum; chan++)
    {
        for (int i = 0; i < blockSize; i++)
```

```
            // just test for scale input data
            dataOut[chan][i] = dataIn[chan][i] * st->gain;
    }
    return 0;
}

int Custom_Algo_Destroy(BlockAlgo *algoSt)
{
    return 0;
}

int Custom_Algo_GetParam(BlockAlgo* algoSt, void* pParam, uint32_t nParamOff, uint32_t *nParamS
{
    memcpy(pParam, algoSt->pParam + nParamOff, *nParamSize);
    for (int i = nParamOff; i < *nParamSize / 4; i++)
        LOGD("param_%d:%f\n", i, algoSt->pParam[i]);
    return 0;
}

int librkstudio_register(void *param)
{
    int ret;
    RkStudioFunc func =
    {
        .algoType        = 0x1101,                      // must same with algoType in custom_a
        .algoInit        = Custom_Algo_Init,
        .algoSetParam    = Custom_Algo_SetParam,
        .algoGetParam    = Custom_Algo_GetParam,
        .algoGetAlgoSize = Custom_Algo_GetAlgoSize,     // You can also set this function as NU
        .algoProcess     = Custom_Algo_Process,
        .algoDestroy     = Custom_Algo_Destroy,
    };

    ret = RKST_ALGO_Register(&func);
    LOGD("register algoType = %d, ret = %d\n", func.algoType, ret);
    return ret;
}
```